ATIVIDADE AULA 3

Prof. Francisco Fambrini (03/10/2026)

Treinamento de detecção de objetos com YOLO (Ultralytics) e dataset do Roboflow Universe

Campo	Preenchimento
Aluno(a)	
Data de entrega	Até 01/11/2026
Modalidade	Individual
Pontuação	0 até 10,0 pontos

1. Objetivo

Colocar em prática o fluxo completo de treinamento de um detector de objetos: escolher e preparar um dataset anotado, treinar uma rede YOLO com a biblioteca Ultralytics, avaliar o resultado e utilizar o modelo treinado em novas imagens.

2. Enunciado

Implemente em Python um código de treinamento utilizando YOLOv8 ou YOLO11 (biblioteca ultralytics). O treinamento deve ser feito com um dataset de imagens escolhido por você no Roboflow Universe:

https://universe.roboflow.com

Ao final, você deve entregar o código-fonte de treinamento, um código-fonte simples de utilização do modelo e o arquivo do modelo treinado, acompanhados das evidências descritas na seção 5.

Requisitos do dataset

- Tarefa de detecção de objetos (anotações com caixas delimitadoras / bounding boxes).
- Entre 2 e 15 classes e entre 300 e 5.000 imagens, para que o treino caiba no tempo de GPU gratuita do Colab.
- Dataset diferente do utilizado como exemplo em aula (leaf disease).
- Exportação no formato YOLOv8 (também compatível com YOLO11), que gera as pastas train, valid, test e o arquivo data.yaml.

Requisitos técnicos

Item	Especificação
Linguagem / biblioteca	Python 3 com a biblioteca ultralytics
Modelo base	yolov8n.pt ou yolo11n.pt (pesos pré-treinados, variante nano ou small)
Ambiente	Google Colab com GPU (Runtime > Alterar tipo de runtime > T4 GPU)
Épocas	Mínimo de 30 (sugestão: 50)
Tamanho de imagem	imgsz = 640

3. Roteiro passo a passo

Passo 1 — Preparar o ambiente

Abra um novo notebook no Google Colab, ative a GPU e instale a biblioteca:

```
!pip install -q ultralytics

import ultralytics
ultralytics.checks()   # confirma versão e GPU disponível
```

Passo 2 — Escolher o dataset no Roboflow Universe

- Acesse **https://universe.roboflow.com** e use a busca (ex.: "helmet", "fruit", "license plate", "pothole").
- Entre na página do dataset e confira: número de imagens, classes, tipo (Object Detection) e exemplos de anotação.
- Clique em **Download Dataset**, escolha o formato **YOLOv8** e a opção **Download zip to computer** (ou "show download code" para baixar via código).
- Anote o link completo do dataset (https://universe.roboflow.com/<workspace>/<projeto>) — ele será exigido na entrega.

Passo 3 — Levar o dataset para o Colab

Opção A — upload do .zip: envie o arquivo pelo painel de arquivos do Colab e descompacte:

```
!mkdir -p /content/dataset
!unzip -q /content/NOME_DO_ARQUIVO.zip -d /content/dataset
```

Opção B — download por código (Roboflow API): o trecho gerado pelo próprio Roboflow em "show download code", no formato abaixo. Use sua chave pessoal e nunca a compartilhe nem a publique no relatório.

```
!pip install -q roboflow
from roboflow import Roboflow
rf = Roboflow(api_key="SUA_CHAVE")
project = rf.workspace("WORKSPACE").project("PROJETO")
dataset = project.version(1).download("yolov8")
```

Passo 4 — Inspecionar o dataset (gera evidências)

Antes de treinar, examine a estrutura e o conteúdo do dataset. Estas saídas fazem parte das evidências obrigatórias:

```
import os, glob
root = "/content/dataset"          # ajuste para a sua pasta

print(open(f"{root}/data.yaml").read())   # conteúdo do data.yaml

for split in ["train", "valid", "test"]:
    imgs = glob.glob(f"{root}/{split}/images/*")
    lbls = glob.glob(f"{root}/{split}/labels/*")
    print(split, "-> imagens:", len(imgs), "| labels:", len(lbls))
```

Em seguida, exiba pelo menos 4 imagens do conjunto de treino com as caixas desenhadas (ou 4 imagens e o conteúdo de um arquivo .txt de label correspondente), para demonstrar que você compreendeu o formato das anotações (classe, x_centro, y_centro, largura, altura, normalizados entre 0 e 1).

Passo 5 — Conferir e ajustar o data.yaml

O data.yaml informa onde estão as imagens e quais são as classes. Verifique se os caminhos de train, val e test existem de fato no Colab. Se necessário, edite para caminhos absolutos:

```
train: /content/dataset/train/images
```

```
val: /content/dataset/valid/images
test: /content/dataset/test/images

nc: 3
names: ['classe_a', 'classe_b', 'classe_c']
```

Passo 6 — Treinar o modelo

Crie o código-fonte de treinamento (arquivo treino.py ou célula do notebook). Estrutura mínima esperada:

```
from ultralytics import YOLO

model = YOLO("yolov8n.pt")        # ou "yolo11n.pt"

results = model.train(
    data="/content/dataset/data.yaml",
    epochs=50,
    imgsz=640,
    batch=16,
    name="atividade_aula4",
)
```

Ao final, o Ultralytics informa a pasta de saída (runs/.../atividade_aula4). Dentro dela, na subpasta weights, estão os arquivos best.pt (melhor modelo na validação) e last.pt (última época). Também são gerados results.png, confusion_matrix.png e os arquivos de curvas de precisão/recall.

Passo 7 — Avaliar o modelo

```
metrics = model.val()   # avalia no conjunto de validação
print("mAP50:", metrics.box.map50)
print("mAP50-95:", metrics.box.map)
```

Observe o gráfico results.png e a matriz de confusão. No relatório, comente em poucas linhas se o modelo aprendeu bem, quais classes tiveram pior desempenho e uma hipótese para o motivo.

Passo 8 — Código simples de utilização

Crie um segundo código-fonte (uso.py), curto, que carrega o best.pt, executa a detecção em uma imagem e salva o resultado:

```
from ultralytics import YOLO

model = YOLO("best.pt")
results = model.predict("minha_imagem.jpg", conf=0.25, save=True)

for r in results:
    for box in r.boxes:
        print(model.names[int(box.cls)], float(box.conf))
```

Teste com pelo menos 3 imagens que não pertençam ao conjunto de treino (podem ser do conjunto de teste ou imagens obtidas por você) e guarde as imagens com as detecções desenhadas.

Passo 9 — Baixar os arquivos e montar a entrega

```
from google.colab import files
files.download("/content/runs/detect/atividade_aula4/weights/best.pt")
# ajuste o caminho conforme a pasta impressa ao final do treino
```

4. Entregáveis

Entregue um único arquivo compactado (.zip) nomeado AULA4_NomeSobrenome.zip contendo:

- ☐ **treino.py** (ou .ipynb) — código-fonte completo de treinamento, comentado.
- ☐ **uso.py** — exemplo simples de utilização do modelo treinado.
- ☐ **best.pt** — modelo obtido no final do treinamento.
- ☐ **Relatório curto (.docx)** — com o link do dataset, as evidências da seção 5 e a análise dos resultados (máx. 4 páginas).

5. Evidências obrigatórias (no relatório)

- ☐ **Link do dataset** no Roboflow Universe (universe.roboflow.com/...), com nome do projeto, número de imagens e de classes.
- ☐ **Conteúdo do arquivo data.yaml** (captura de tela ou texto colado), mostrando caminhos, nc e names.
- ☐ **Estrutura das pastas** do dataset (train/valid/test, com subpastas images e labels) e contagem de arquivos de cada uma — saída do código do Passo 4.
- ☐ **Imagens do dataset escolhido** — pelo menos 4 exemplos do conjunto de treino com as anotações.
- ☐ **Log do treinamento** — captura mostrando as últimas épocas e a tabela de métricas por classe.
- ☐ **Gráficos gerados** — results.png e confusion_matrix.png.
- ☐ **Resultados de inferência** — pelo menos 3 imagens com as detecções do modelo treinado.

6. Dicas e problemas comuns

- **Erro de caminho no data.yaml:** é o problema mais frequente. Confira se as pastas train/images, valid/images e test/images existem e use caminhos absolutos.
- **Falta de memória na GPU:** reduza batch (por exemplo, para 8) ou imgsz (para 512).
- **Métricas muito baixas:** aumente as épocas, confira se as classes estão equilibradas e se as anotações estão corretas.
- **Sessão do Colab encerrada:** salve a pasta runs no Google Drive (montando o Drive) para não perder o treinamento.
- **Chave da API do Roboflow:** não deixe sua chave exposta no código entregue; substitua por um texto genérico antes de enviar.
- **Licença do dataset:** verifique a licença indicada na página do dataset no Roboflow e cite a fonte no relatório.